SANKUR KUNDU

Curriculum Vitae | ML Systems Engineer & Independent Researcher

Bhubaneswar, India (open to relocation) | +91 97761 94201 | sankur.kundu.tw@gmail.com

linkedin.com/in/sankur-kundu | github.com/Shakhtar-Sankur | shakhtar-sankur.github.io | evidence for every claim

SUMMARY

ML systems engineer and independent researcher (B.Tech CSE, 2026), sole author of four manuscripts, and co-founder of Gigzen, with fixes merged into **Meta ExecuTorch** and **Hugging Face OpenEnv** and PRs in review at **Google JAX**, **NVIDIA**, **Apple MLX**, **vLLM**, **PyTorch** and research labs at **MIT** and **Oxford, UCL & Imperial**. I build systems from scratch and measure them against the best in the field: GPU kernels at **cuBLAS parity**, an ML compiler faster on CPU than **torch.compile, ONNX Runtime and JAX/XLA**, an LLM engine at **1.05×** vLLM's throughput with identical output, a GPU quantum decoder **bit-identical to IBM's**, a tokenizer that beats **GPT-4o's and Gemini's** in 23 of 24 Indian languages, and a collision-probability engine that agrees with NASA CARA's on all **53** of NASA's test conjunctions at **15–19×** its speed. Every number below is reproducible from scripts in a public repository.

RECOGNITION

- **NASA CARA** (Conjunction Assessment Risk Analysis): its operations team lead referred my independent reproduction of CARA's Monte Carlo results on all **53** of its test cases to CARA's analysis team, which will report its findings (20 Oct 2026).
- **Entrepreneur First** (UK): scouted by its Talent Investor from my public work alone (Oct 2026), citing my **cuBLAS-parity** compiler.
- **Meta ExecuTorch** and **Hugging Face OpenEnv**: maintainers merged three of my fixes (**#23677**, approved by two Meta maintainers; **#1441** and **#1442**, reviewed by Hugging Face; 16–19 Oct 2026).

SCALE AND REACH

- **Computation**: **531 million** orbit pairs screened in **28 s** on one GPU; **10,000** simulated database clusters through **610,064** crashes.
- **Reach**: **15** open-source systems with kernels for NVIDIA GPUs, Apple Metal and Google TPUs; Populace **1,600+** npm downloads.

MANUSCRIPTS AND RELEASES

- S. Kundu. "Removing the Token Tax from an Existing Model: A Vocabulary Extension for 24 Indian Languages That Keeps Every Other Token." Manuscript, 2026; arXiv (cs.CL) submission in progress (**paper**).
- S. Kundu. "The Sampler Was Not the Policy: Finding and Removing a Silent Training–Inference Mismatch in a From-Scratch GRPO Stack." Technical report, 2026; arXiv (cs.LG) submission in progress (**paper**).
- S. Kundu. "Reproducing NASA CARA's Monte Carlo Probability of Collision: An Open Two-Body Implementation, Checked on All 53 of CARA's Test Conjunctions." Manuscript, 2026; arXiv (physics.space-ph) submission in progress (**paper**).
- S. Kundu. "Scam Defence Stops at English: Measuring the Multilingual Protection Gap and Linking Scam Campaigns Across Languages." Manuscript in preparation, 2026.
- Released **lipi-Indic 96K** on **Hugging Face** (Apache-2.0): 24 Indian languages, English and code tokens unchanged.

OPEN SOURCE CONTRIBUTIONS

- **Meta ExecuTorch #23677** (**merged**, approved by two maintainers): the 2-bit and 4-bit quantized embedding kernels used by on-device LLMs read memory out of bounds on an invalid token id; added the bounds check the 8-bit kernel has, with tests that fail without it.
- **Hugging Face OpenEnv #1441**, **#1442** (**both merged**; RL environments): composite reward rubrics ran pre-hooks after scoring their children (a timer read **0.01 ms** for **400 ms** of work), and one delegating to an async judge recorded a coroutine as its score; **9** of **13** new tests fail without the fixes.
- **Google JAX #41859** (under review): reverse-mode autodiff for lax.reduce_window with custom computations, open since 2021; **NVIDIA Ising-Decoding #98** (under review): tests accept NVIDIA's published models.
- **Apple MLX #4737** (under review): a data race in the distributed ring backend's socket threads, which read their task queues outside the lock; ThreadSanitizer reports **2–4** races per run before the fix and **0** after, and the 8-rank ring tests pass.
- **Meta ExecuTorch #23912** (under review): the same kernels checked zero points against the output type instead of the scales' type, so valid fp16-scale models with fp32 output aborted, and the .out variants read past the end of their scales; **6** new tests fail without the fix, **86 of 86** pass with it.
- **vLLM #60955** (under review): against a float64 reference on a T4, the FlashInfer prefill tests compared non-causal attention with a causal reference and passed only on a loose tolerance; with the fix, **32 of 32** cases pass and **all 32** fail if the bug returns.
- **MIT Han Lab nunchaku #957** (under review): compose_lora applied a single 4-bit LoRA at full strength for any strength below 1, so 0 did not switch it off; **3 of 4** new tests fail without the fix.
- **BOLD Lab (Oxford, UCL, Imperial) JaxMARL #206** (under review): the wrapper behind 8 of its Q-learning baselines blanked the first observation of every episode after an auto-reset; fixed, with tests that fail without it.

- **PyTorch #199875**, **#199441**, **#199646** (under review): deterministic torch.empty under torch.compile, where an earlier fix was reverted; **13** stale Inductor skips removed, **8** determinism tests re-enabled.

PUBLIC-INTEREST AI AND SPACE SAFETY

babelguard — Python: an open early-warning system for scams that cross languages (in progress) · study page

- Measured the largest open set of reported scam SMS (**33,869** in **49** languages): South Asian languages are **0.8%** of it and African **0.1%**; **96** scam templates recur across languages, and genuine messages exist for only **4 of 49** languages.
- Measured the protection gap: an English keyword filter catches **86%** of English scams but **10–36%** of Korean, Japanese, Urdu, Arabic, Thai and Hindi ones, and Bengali scams **38%** of the time in Bengali script against **77%** in Latin letters.
- Built linking by meaning that recognises a translated scam **85%** of the time (text matching: **1.5%**) and a crash-safe streaming engine: **4,057 messages/s** on 4 cores, **0.21 KB** per message at 1M, nothing lost in **50** kills.

lipi — Python and PyTorch: cutting the token tax on Indian languages · study page

- Measured the token tax of 24 Indian languages on 15 tokenizers (Odia: **4.99×** English on GPT-4o); built **lipi-Indic** (on Hugging Face): fewer tokens than **GPT-4o's and Gemini's** in **23 of 24** languages (**1.30×** vs 1.92×, 1.58×).
- Its 96K new merges fire only on Indian-script bytes, so English, code and other scripts keep exactly their Qwen 2.5 token ids; Santali, the costliest language measured (**13.7×** English on GPT-4o), drops to **1.52×**.
- Extended Qwen2.5-0.5B's vocabulary and continued pre-training on 2× T4 (DDP, ZeRO-1): Odia takes **8.6× fewer** tokens, and in equal GPU time the model predicts held-out Odia **20% better**, with Odia→English chrF++ 16.0 → 24.8.

kessler — C++/CUDA and Python: an independent auditor of space traffic · live globe

- Built SGP4 and all-vs-all conjunction screening in C++/CUDA: all **531 million** pairs of the **32,593** tracked objects a day ahead in **28 s** on a T4, matching CelesTrak SOCRATES to **0.24 m** (median).
- Built a collision-probability Monte Carlo that agrees with NASA CARA's on all **53** of NASA's test conjunctions (χ^2 p = 0.67), including the **28** where the standard 2D method is wrong, and runs **15–19×** faster per core than NASA's own library.
- Runs in public every day: a GitHub Actions pipeline screens every active satellite from CelesTrak's open catalogue (**16,768** satellites, about **60,000** close approaches a day) and publishes a 3D globe; on the GPU, SGP4 computes **125.7M** orbit states/s in fp64, within **1e-10 km** of Vallado's reference.
- Built a maneuver detector, with debris as its control group, to audit operators' collision-avoidance reports.

ML SYSTEMS: TRAINING, INFERENCE AND RL

ratchet — Python, PyTorch and C++/CUDA: reinforcement-learning post-training (GRPO) from scratch · study page

- Built GRPO post-training end to end: rollouts with per-token log-probabilities on relay (through a C API), a PyTorch trainer data-parallel on tandem, in-place GPU weight sync, partial rollouts and one-step-ahead training.
- On 2× T4, 100 steps lifted Qwen2.5-0.5B on GSM8K from **48.1% to 52.5%** (all 1,319 test problems); weight sync is **369×** faster than a reload, and overlapping generation with training makes steps **1.9×** faster.
- Found that the engine's fp16 weights rounded away most updates, so it sampled a policy the trainer never had (gap up to 5.2 nats); mixed-precision rounding cut it to **0.04**, and sync rounding outweighs fp16 arithmetic **9–22×**.

relay — C++, CUDA and Swift: a disaggregated, fault-tolerant LLM inference cluster · study page

- Built C++/CUDA prefill and decode workers on custom flash-attention and flash-decoding kernels, a KV-cache transfer engine that streams each layer while the next computes, and a Swift control plane with prefix-aware routing.
- On a T4, cut 2,048-token prefill **8.7×** (3.7 s → 0.43 s) and batch-32 decode **5.4×**; streaming hides **99–100%** of the KV transfer. Output matches Hugging Face and stays bit-identical when batched, cached or split across workers.
- A gather kernel packs each layer's fp16 KV pages into pinned memory on a separate copy stream, ordered by per-layer CUDA events: a 46 MB transfer that cost 21.6 ms after prefill costs **0.08 ms** when streamed.
- Fault-tolerant: watchdogs catch killed, frozen or stalled workers and their requests resume elsewhere with identical output (**384/384** through 60 kills; no request lost in **300** fault-injection seeds).

switchyard — CUDA, C++ and Python: mixture-of-experts serving with expert parallelism · study page

- Served OLMoE-1B-7B (64 experts, top-8) across 2× T4 with expert parallelism built from scratch: CUDA kernels for routing, sorting and grouped expert GEMM/GEMV on tensor cores, and peer-to-peer dispatch in place of NCCL.
- Decodes **2.2× faster** than Hugging Face (112 vs 52 tokens/s at batch 8), bit-identical to one GPU; a decode MoE layer runs **5.4–7×** faster than a per-expert loop, and dispatch is **1.25×** faster than NCCL all-to-all at 256 tokens/GPU.
- Measured on real text: dropping tokens at full experts, as capacity-limited routing does, raises perplexity from **12.9 to 20.2** even at capacity factor 2, so the kernels never drop one. Found and fixed two cross-stream races between the GPUs.

crucible — Python on Linux: sandboxed environments for RL on coding agents · study page

- Built sandboxes from Linux namespaces, seccomp-BPF and limits; a live one forks in **4.5 ms** vs 138 ms to rebuild, so rollout trees branch from snapshots with up to **10.9×** less environment time than replaying. Its OpenEnv-compatible grader passes **0 of 9** reward hacks (the usual grader: 4).
- GRPO on ratchet with every reward from a sandbox: MBPP pass@1 **34.9% → 38.6%** (3 seeds; paired test, **p = 0.013**).

tandem — Python, C++ and CUDA: distributed training from scratch, without torch.distributed or NCCL · study page

- Built DDP, ZeRO-1/2/3 (FSDP-style sharding), GPipe and 1F1B on a custom ring all-reduce; a C++/CUDA engine (lock-free shared-memory rings, CUDA IPC, a fused reduce kernel) runs all-reduce **1.6–2.8×** faster than the Python one.

- On 2× T4, DDP and ZeRO-1/2/3 train **bit-identically** to PyTorch DDP/FSDP on NCCL, with all-reduce within **2%** of NCCL's bandwidth at 64–256 MB; on an 86M-parameter GPT, ZeRO-3 cuts per-GPU memory 17%.

QUANTUM ERROR CORRECTION

syndrome — C++ and CUDA: an open GPU decoder for IBM's quantum LDPC codes · study page

- Implemented IBM's **Relay-BP** decoder for the **[[144,12,12]] gross code** (1,728 detectors, 67,752 error mechanisms, 391,320 edges) from one C++/CUDA source; on a T4 it matches IBM's Rust reference **bit for bit** on **256 of 256** shots, every float64 and float32 posterior included, up to 301 relay legs.
- Decodes a single shot **28.7×** faster than IBM's decoder (median **37.5 ms** vs 1.08 s, every shot identical) on the whole GPU, and a batch **4.0×** faster than IBM's on all CPU threads; IBM's published error rate reproduced first (**41 vs 44** in 10,000).

parity — Python, Stim and PyTorch: an independent benchmark of quantum error-correction decoders

- Ran NVIDIA's two published **Ising** pre-decoders through NVIDIA's own pipeline (d = 5–13, **524,288** shots each), matching its logged error rates to every printed digit, against uncorrelated and correlated **PyMatching** on identical samples.
- Reproduced **PyMatching 2 / Sparse Blossom** (Higgott & Gidney): decode-time scaling exponent **1.38** (paper: 1.32), surface-code threshold crossings at **0.61–0.80%** for d = 3–11, and **0** logical errors in 1M shots per basis at d = 17.

COMPILERS AND GPU KERNELS

tessel — Rust, zero dependencies: a tile language and compiler for GPU and TPU kernels · study page

- Designed a Triton-style tile language and wrote its compiler: typed tile IR, linear layouts, CUDA for tensor cores (ldmatrix, mma.sync, swizzled shared memory, software pipelining), an auto-tuner, and Metal and TPU (Pallas) backends.
- On a T4, checked against fp64 references: GEMM at **parity with cuBLAS** (1.00–1.04×), causal attention **1.3–1.9×** faster than PyTorch SDPA, RMSNorm 1.27× faster than torch.compile, softmax 1.07× faster than Triton.
- Built an LLM engine on its kernels alone (paged KV cache, continuous batching, CUDA graphs): on TinyLlama-1.1B its output matches **vLLM** token for token at **1.05×** vLLM's throughput with 32 sequences.

kiln — Rust, zero dependencies: an ML compiler from ONNX to SIMD C and CUDA · study page

- Reads ONNX, optimizes and fuses the graph, generates and compiles SIMD C and CUDA (NVRTC) code, and auto-tunes matrix multiplies on the target machine; its output matches PyTorch's.
- On 4 CPU cores, the lowest median latency on all three benchmark models: BERT in **117 ms** against 149 for torch.compile, 153 for ONNX Runtime and 189 for JAX/XLA; on a T4, SmolLM2 **1.65×** faster than torch.compile.

DISTRIBUTED SYSTEMS AND VERIFICATION

quorumdb — Rust, zero dependencies: a distributed SQL database, PostgreSQL wire-compatible · study page

- Built a distributed SQL database that psql connects to: LSM storage, Raft and self-rebalancing multi-Raft, serializable ACID transactions (Percolator) and a SQL planner; **1,500 tpmC** on a TPC-C-derived benchmark.
- Verified by deterministic simulation (**10,000** Raft clusters, 610K crashes, histories machine-checked) and a TLA+ model (337K states): found **6 real bugs** and catches all 15 planted ones.

faultline — Rust, zero dependencies: Jepsen-style fault injection for etcd

- Runs a real cluster in network namespaces under partitions, kill -9 and SIGSTOP, and checks histories for linearizability: **1.26M** operations under **288** faults across 4 etcd release lines; traced lost updates under etcd locks to a paused leader applying a timed-out write **6 s** late; fenced writes lost none.

EXPERIENCE

Gigzen — Co-founder & Founding Engineer **Apr 2026 – Present**

Early-stage startup (company name approved by India's MCA; incorporation in progress) · Sole engineer across every product

- Sole engineer on **Waggle**, a delivery platform of three Android apps (riders, shops, customers) from one TypeScript/React/Capacitor codebase on Supabase PostgreSQL: **53 tables, 90 RLS policies, 154 SQL functions**.
- Put first-accept-wins dispatch, UPI reconciliation and GST billing in the database, where no client can bypass them.
- Security-tested it with **150 automated attacks** on its database access rules (**all blocked**) and a 42-step Playwright run.
- Built **Populace**, an open-source tool that fills an app with simulated users acting through its real API (**npm**, **1,600+ downloads**); its first run found **5 defects in 3.5 minutes** after manual testing had passed.

Freelance Data Scientist — Upwork, remote (client in Spain) **Jun 2026 – Jul 2026**

- Merged two hotel censuses (5,909 and 8,058 records) and built a Google Trends panel of **10,038 city-months** in Pandas.

SKILLS

GPU and compilers: CUDA (tensor cores, mma.sync, ldmatrix, graphs), NVRTC, Triton-style tile languages, Metal, Pallas, SIMD
ML systems: GRPO, DDP, ZeRO-1/2/3, FSDP, 1F1B, MoE expert parallelism, paged KV cache, speculative decoding
Distributed systems: Raft, multi-Raft, MVCC, Percolator, LSM trees, deterministic simulation, TLA+, fault injection
Languages and platforms: C++20, CUDA, Rust, Python, Swift, TypeScript, SQL; PyTorch, JAX, PostgreSQL, Docker, Kubernetes

EDUCATION

B.Tech, Computer Science and Engineering (AI & ML) — IEM Kolkata **Sep 2022 – Apr 2026**